

Alberta Electricity Live Dashboard

A plain-language guide to what this project does and how live API data actually works — written for the first time you're building a pipeline like this.

Project	AESO Live Electricity Market Dashboard
Files	AESO_Live_Electricity_Dashboard.ipynb, aesodb.sql, this PDF
Data source	AESO (Alberta Electric System Operator) Public API
Stack	Python (Jupyter) → SQLite/SQL → Power BI

1. What is an API, actually?

Every project you've built so far started with a file already sitting on your computer — a CSV you downloaded once, a Kaggle dataset. An API (Application Programming Interface) is different: it's a live door into someone else's database. Instead of downloading a file, your code sends a request over the internet, and the other computer sends back an answer, right now, in the format the data currently exists in.

Think of it like ordering at a restaurant counter instead of raiding the fridge yourself. You don't get to walk into AESO's database — you ask for something specific ("give me pool prices from July 1 to August 1"), and they hand you exactly that, freshly prepared, every time you ask.

The four things every API call needs

- **Endpoint** — the web address you're asking. Ours is `https://apimgw.aeso.ca/public/poolprice-api/v1.1/price/poolPrice`
- **Method** — almost always **GET** when you're just reading data (as opposed to POST, which is for sending/creating data)
- **Headers** — metadata sent alongside the request. This is where your API key goes, proving to AESO who's asking
- **Parameters** — the specifics of your request, like `startDate` and `endDate`

The response comes back as **JSON** — a nested, text-based format that looks like a mix of Python dictionaries and lists. Pandas' `json_normalize()` function is what flattens that nested structure into the familiar rows-and-columns table you're used to working with.

2. Why this project, specifically

Your resume currently has zero live-data projects and zero projects tied to energy or construction — the two lower-competition sectors you and I identified around Edmonton. This project directly closes that gap, and it teaches a skill (scheduled live ingestion) that almost no entry-level portfolio has, because most tutorials use static CSVs.

AESO is the non-profit that operates Alberta's entire electricity grid and wholesale market. Every hour, it publishes the market-clearing **pool price** — what generators get paid and what retailers/large consumers pay — plus **Alberta Internal Load**, the total electricity demand across the province. This is real market data used by actual energy traders and analysts.

3. The pipeline, step by step

Step 1 — Get an API key (5 minutes, free)

- Go to `developer-apim.aeso.ca/signup` and register with your email
- Confirm your email, then sign in
- Click **Products** (top right) → subscribe to **AESO Public API**
- Click **Show** next to your new key and copy it — you'll paste it into the notebook when prompted

Treat this key like a password. It's tied to your email and identity. The notebook uses Python's `getpass` module so the key is typed privately and never gets saved inside the notebook file itself.

Step 2 — Pull the data (the notebook)

`AESO_Live_Electricity_Dashboard.ipynb` sends a GET request to two AESO endpoints — one for price, one for demand — for the last 30 days, and converts each JSON response into a pandas DataFrame. If you don't have a key yet, the notebook can generate realistic sample data with the identical structure, so you can build and test everything else first.

Step 3 — Clean and merge

Price and demand come from two separate endpoints, so they're joined on their shared hourly timestamp. Duplicate timestamps are dropped, and any isolated missing prices (AESO occasionally has brief reporting gaps) are forward-filled for up to 3 hours.

Step 4 — Flag anomalies

This reuses the anomaly-detection logic from your Financial Risk & Portfolio project: a rolling 7-day mean and standard deviation, converted into a z-score. Any hour where price sits more than 2.5 standard deviations above its recent trend gets flagged as a spike — a genuinely useful signal, since Alberta's deregulated market can jump from \$30/MWh to \$500+/MWh within a single hour during supply shortages.

Step 5 — Export to SQL

The notebook writes the cleaned table into a local SQLite database (`aeso_electricity_data.db`) using `pandas.to_sql()`. SQLite is a real, fully-functional SQL database that lives in a single file — perfect for a portfolio project since it needs no server setup, but the SQL you write against it is standard SQL that transfers directly to PostgreSQL or SQL Server.

```
conn = sqlite3.connect('aeso_electricity_data.db')
df.to_sql('electricity_prices', conn, if_exists='replace', index=False)
```

Open `aeso_sql_setup.sql` to see the table schema plus ready-to-run queries: daily summaries, average price by hour of day, top price-spike hours, and a demand-vs-price correlation check.

Step 6 — Connect Power BI

The simplest path (recommended for a first live-data project):

- Open Power BI Desktop → **Get Data** → **Text/CSV** → select `aeso_electricity_data.csv`
- In Power Query, confirm `time_local` is typed as Date/Time and price/demand columns are typed as Decimal Number
- Load, then build: a line chart of price over time, a KPI card for spike count, a bar chart of average price by hour

If you want the SQL practice reps instead: **Get Data** → **ODBC**, using a SQLite ODBC driver pointed at `aeso_electricity_data.db`. This is more setup but lets you say in an interview that you connected Power BI directly to a SQL database rather than a flat file.

4. Making it actually live (not just a one-time pull)

Right now, re-running the notebook manually gives you fresh data each time — that's already "live" in the sense that matters for a portfolio. To make it automatic:

- **Windows:** Task Scheduler → create a task that runs `jupyter nbconvert --to notebook --execute --inplace AESO_Live_Electricity_Dashboard.ipynb` on a schedule (e.g. hourly)
- **Mac/Linux:** the same command via `cron`
- **Power BI:** if your CSV is refreshed automatically and Power BI is pointed at a fixed file path, use Power BI's built-in **Scheduled Refresh** to pull the latest version on a timer

5. How to talk about this in an interview

This project gives you a concrete answer to "tell me about a time you worked with real-time or automated data" — a question static-CSV projects can't answer well. A tight version:

"I built an automated pipeline pulling live Alberta electricity market data from AESO's public API — hourly pool prices and system demand — cleaned and merged it in Python, flagged abnormal price spikes using a rolling z-score, and pushed it into a SQL database that feeds a Power BI dashboard. It's scheduled to refresh automatically, so the dashboard reflects current market conditions rather than a static snapshot."

If asked to go deeper: be ready to explain *why* a z-score threshold of 2.5 was chosen (a common statistical convention for flagging outliers — roughly the top ~0.6% of a normal distribution), and why SQLite was the right call for a portfolio project versus a hosted database (zero setup cost, fully portable, real SQL underneath).

Companion files: AESO_Live_Electricity_Dashboard.ipynb (the pipeline) and aesosql_setup.sql (schema + queries). Run the notebook first — it produces the CSV and SQLite file the SQL and Power BI steps depend on.